

IFT436 - Série d'exercices #7 : programmation dynamique

Manuel Lafond

Exercice 1: Considérez l'algorithme pour trouver un ensemble indépendant de taille maximum dans un arbre. La version des notes de cours ne fait que retourner la taille, mais ne donne pas d'ensemble indépendant. Écrivez une procédure permettant de retrouver un ensemble indépendant concret au lieu de seulement sa taille. (recommandé: calculez M_0 et M_1 comme dans les notes de cours, puis commencez de la racine jusqu'aux feuilles pour construire une solution).

On suppose qu'on a calculé M_0 et M_1 tel que vu en classe. Une fois ces valeurs connues pour chaque sommet, on parcourt l'arbre de haut en bas pour reconstruire une solution S . L'idée est d'ajouter un sommet v à S seulement si $M_1[v] \geq M_0[v]$. De plus, on interdit d'ajouter v si son parent a déjà été ajouté.

```
fonction getIndSet( $T, r, M_0, M_1$ )  
  //  $r$  est la racine courante  
  //  $S$  est une variable globale, initialement vide  
  si  $M_1[r] \geq M_0[r]$  et ( $r$  est la racine de  $T$  ou  $r.parent \notin S$ )  
    alors  
      Ajouter  $r$  à  $S$   
    pour chaque enfant  $v_i$  de  $r$  faire  
      getIndSet( $T, v_i, M_0, M_1$ )  
  fin  
  return  $S$ 
```

Exercice 2: Soient S et T deux chaînes de caractères, disons sur les lettres

de a à z . On veut transformer S en T à l'aide de trois opérations possibles: on a le droit de supprimer un caractère, insérer un caractère (n'importe où) ou bien remplacer un caractère.

Par exemple, on peut passer de $S = algo$ à $T = magma$ en quatre opérations: successivement, on aurait

$$algo \rightarrow malgo \rightarrow mago \rightarrow magmo \rightarrow magma$$

(on a appliqué, dans l'ordre, une insertion, une suppression, une insertion et un remplacement)

Donnez un algorithme qui calcule le nombre minimum d'opérations nécessaires pour passer de S à T .

Indice: calculez une table $D[i, j]$ qui exprime le nombre d'opérations pour passer de la sous-chaîne $S[1..i]$ à la sous-chaîne $T[1..j]$, puis exprimez $D[i, j]$ avec une récurrence.

Soit n la longueur de S et m la longueur de T . Soit $D[i, j]$ le nombre d'opérations minimum pour passer de la chaîne $S[1..i]$ à la chaîne $T[i..j]$, où $i \in \{1, 2, \dots, n\}$ et $j \in \{1, 2, \dots, m\}$. L'idée est de calculer $D[i, j]$ à partir de $D[i - 1, j]$, $D[i - 1, j - 1]$ et $D[i, j - 1]$. Donc si on voit D comme une matrice, on regarde les cellules adjacentes à $D[i, j]$ pour la calculer. La valeur recherchée est $D[n, m]$.

Pour voir le fonctionnement, fixons notre attention sur les caractères $S[i]$ et $T[j]$ et observons qu'il y a trois cas possibles dans une séquence d'opérations optimale:

- a. le caractère $S[i]$ a été supprimé. Dans ce cas, on peut prendre le meilleur scénario qui transforme $S[1..i-1]$ en $T[1..j]$, puis effectuer la suppression à la fin. Quand ce scénario s'applique, on a $D[i, j] = D[i - 1, j] + 1$.
- b. le caractère $T[j]$ a été inséré. Dans ce cas, on peut prendre le meilleur scénario qui transforme $S[1..i]$ en $T[1..j-1]$, puis effectuer l'insertion à la fin. Quand ce scénario s'applique, on a $D[i, j] = D[i, j - 1] + 1$.
- c. $S[i]$ n'a pas été supprimé et $T[j]$ n'a pas été inséré. Dans ce cas, on peut prendre le meilleur scénario qui transforme $S[1..i-1]$ en $T[1..j-1]$. Ensuite, si $S[i] = T[j]$, on ne fait rien et $D[i, j] = D[i - 1, j - 1]$. Si $S[i] \neq T[j]$, on doit faire une modification et on a $D[i, j] = D[i - 1, j - 1] + 1$.

En ajoutant des cas de base pour $i = 0$ et $j = 0$, on obtient donc la récurrence suivante. Soit $\delta_{i,j} = 0$ si $S[i] = T[j]$, et $\delta_{i,j} = 1$ si $S[i] \neq T[j]$.

$$D[0, j] = j \quad \forall j \text{ (car il faut } j \text{ insertions)}$$

$$D[i, 0] = i \quad \forall i \text{ (car il faut } i \text{ suppressions)}$$

$$D[i, j] = \min(D[i - 1, j] + 1, D[i, j - 1] + 1, D[i - 1, j - 1] + \delta_{i,j})$$

Il ne reste qu'à implémenter cette récurrence. Ceci donne ce qu'on appelle l'algorithme de Needleman-Wunsch.

```
fonction  $nw(S, T)$ 
  //  $n = |S|, m = |T|$ 
   $D$  = matrice de dimension  $n \times m$ 
  pour  $j = 1..m$  faire
     $D[0, j] = j$ 
  fin
  pour  $i = 1..n$  faire
     $D[i, 0] = i$ 
  fin
  pour  $i = 1..n$  faire
    pour  $j = 1..n$  faire
       $\delta_{i,j} = 0$ 
      si  $S[i] \neq T[j]$  alors
         $\delta_{i,j} = 1$ 
       $D[i, j] =$ 
         $\min(D[i-1, j] + 1, D[i, j-1] + 1, D[i-1, j-1] + \delta_{i,j})$ 
    fin
  fin
  return  $D[n, m]$ 
```

Cet algorithme prend un temps $O(nm)$.

Comme d’habitude, ceci ne permet pas d’avoir un scénario d’édition concret. On peut en reconstruire un à partir de D en commençant à $D[n, m]$ et en demandant, à chaque étape, ce qui a donné lieu à l’optimal. Nous n’allons pas retourner une séquence d’opérations à proprement dit. Nous allons plutôt retourner le traitement à effectuer sur chaque caractère de S et T . Une séquence d’opérations peut ensuite être reconstruite.

Il est recommandé de faire un exemple pour bien comprendre la reconstruction. Une autre option est de lire la version “alignement” de Needleman-Wunsch: https://en.wikipedia.org/wiki/Needleman%E2%80%93Wunsch_algorithm.

```

fonction nwbacktrack( $D, S, T$ )
  //  $n = |S|, m = |T|$ 
   $E = ()$  // séquence d’opérations à retourner
   $i = n, j = m$ 
  tant que  $i > 0$  et  $j > 0$  faire
    si  $i > 0$  et  $D[i, j] = D[i - 1, j] + 1$  alors
      //  $D[i, j]$  a été obtenu avec une suppression de  $S[i]$ 
       $E.prepend(\text{“supprimer } S[i]\text{”})$ 
       $i --$ 
    sinon si  $j > 0$  et  $D[i, j] = D[i, j - 1] + 1$  alors
      //  $D[i, j]$  a été obtenu avec une insertion de  $T[j]$ 
       $E.prepend(\text{“insérer } T[j] \text{ après } S[i]\text{”})$ 
       $j --$ 
    sinon
      si  $S[i] \neq T[j]$  alors
         $E.prepend(\text{“transformer } S[i] \text{ en } T[j]\text{”})$ 
      sinon
         $E.prepend(\text{“matcher } S[i] \text{ avec } T[j]\text{”})$ 
       $i --$ 
       $j --$ 
  fin
  return  $E$ 

```

Exercice 3: Considérez le problème du sous-tableau contigu maximum de la série précédente (trouver un sous-tableau $[t_i, t_{i+1}, \dots, t_j]$ de somme maximum

dans un tableau T). Donnez un algorithme en temps $O(n)$ de programmation dynamique pour ce problème.

Soit $D[i]$ la valeur maximum d'un sous-tableau contigu de $T[1..i]$ sous la contrainte que $T[i]$ est dans ce sous-tableau (ce qu'on appelle un sous-tableau suffixe). On remarque qu'il y a 2 façons d'avoir ce sous-tableau: soit on prend le meilleur sous-tableau suffixe de $T[1..i-1]$ et on ajoute l'élément $T[i]$, on bien on ne fait pas ça. Dans ce dernier cas, on doit alors prendre seulement $T[i]$ comme sous-tableau. On retourne le $D[i]$ maximum, parce que le sous-tableau de somme maximum doit bien terminer quelque part, et on a calculé toutes les possibilités. Ceci donne lieu à l'algorithme suivant.

```
fonction sousTabMax( $T$ )  
  si tous les éléments de  $T$  sont négatifs alors  
    return 0    // La meilleure solution ne prend rien  
   $D = [0] * n$   
   $D[1] = T[1]$   
  pour  $i = 2..n$  faire  
     $D[i] = \max(D[i-1] + T[i], T[i])$   
  fin  
  return  $\max_{i \in \{1, \dots, n\}} (D[i])$ 
```

Ceci prendra un temps $O(n)$.

En exercice: reconstruisez une solution concrète à partir de D . Autre exercice, faites tout ça mais avec espace supplémentaire $O(1)$.

Exercice 4: Vous avez un commerce de vente de bâtons de bois, et le prix d'un bâton dépend de sa longueur. On vous donne un tableau de prix T , où $T[i]$ représente le prix de vente d'un bâton de longueur i . Vous avez un bâton de bois de longueur ℓ que vous pouvez couper en plusieurs morceaux de tailles différentes. Vous voulez trouver le découpage qui va maximiser la somme des prix de vente des sous-bâtons.

Par exemple, si $T = [1, 3, 5, 7]$ et $\ell = 10$, vous pouvez couper votre bâton en 3 morceaux: deux de taille 3 et un de taille 4, pour un prix total de 17. Vous pourriez aussi couper deux bâtons de taille 4 et un de taille 2, totalisant aussi 17.

Donnez un algorithme qui calcule le prix de vente maximum que l'on peut atteindre étant donné un tableau de prix T et un bâton de longueur ℓ . La complexité de votre algorithme peut dépendre de $|T|$ et ℓ .

Soit $P[i]$ le prix de vente maximum que l'on peut atteindre avec un bâton de longueur i . On cherche la valeur de $T[\ell]$. On suppose $T[j] = \infty$ s'il n'y a pas de prix donné pour une longueur j . On suppose aussi $P[i] = \infty$ pour tout $i \leq 0$. Il devrait être facile de voir que la récurrence suivante s'applique:

$$P[i] = \max(T[i], \max_{k \in \{1, \dots, |T|\}} (P[i - k] + T[k]))$$

Ceci correspond à tester toutes les façons de faire une première coupure de longueur k , et de prendre la meilleure solution pour un bâton de longueur $i - k$. Cette récurrence peut se calculer avec deux boucles imbriquées allant de 1 à ℓ , et l'autre de 1 à $|T|$, ce qui prendra un temps $O(\ell \cdot |T|)$. Le code complet sera donné en classe.

Exercice 5: On se rappelle du problème de la sous-somme: on reçoit en entrée un tableau d'entiers $S = \{s_1, \dots, s_n\}$ et une somme cible t , et on veut savoir s'il existe un sous-ensemble $S' \subseteq S$ tel que $\sum_{s \in S'} s = t$.
Donnez un algorithme en temps $O(n^2t)$ qui résout ce problème.

Indice: ceci est similaire au problème de la monnaie, mais on a le droit d'utiliser chaque élément de S une seule fois. Une façon est de calculer une table D , où $D[k, j]$ exprime si oui ou non on peut atteindre une somme de j en utilisant seulement les éléments $\{s_1, \dots, s_k\}$. Trouvez une récurrence permettant de calculer D .

Soit $D[k, j]$ un booléen qui se met à *true* s'il existe $S' \subseteq \{s_1, \dots, s_k\}$ tel que $\sum_{s \in S'} s = j$. On remarque qu'il y a deux cas possibles quand $D[k, j] = \text{true}$. Soit (1) on n'a pas besoin de s_k , et donc $D[k - 1, j] = \text{true}$; ou soit (2) on a besoin de s_k , et donc $D[k - 1, j - s_k] = \text{true}$. On a donc la récurrence

$$D[k, j] = D[k - 1, j] \vee D[k - 1, j - s_k]$$

Une fois tous les D calculés, il ne reste qu'à retourner $D[n, t]$.

```

fonction sousSomme( $S, t$ )
   $D = \text{vide}$  // On suppose  $D[k, j] = \text{false}$  par défaut
  pour tout  $k, j$ , incluant  $j, k < 0$ 
  pour  $k = 1..n$  faire
    pour  $j = 1..t$  faire
      si  $j = s_k$  alors
         $D[k, j] = \text{true}$ 
      sinon
         $D[k, j] = D[k - 1, j] \vee D[k - 1, j - s_k]$ 
      fin
    fin
  fin
  return  $D[n, t]$ 

```

Cet algorithme est pseudo-polynomial et prend un temps $O(nt)$.

Exercice 6: Soient A et B deux matrices, respectivement de dimensions $n \times m$ et $m \times p$. Le produit AB donne une matrice C de dimension $n \times p$, où $c_{i,j}$ est défini par $\sum_{k=1}^m a_{i,k} \cdot b_{k,j}$. Le calcul de toutes les entrées de C nécessite $n \cdot m \cdot p$ opérations.

Supposons maintenant qu'on veut calculer le produit de k matrices, disons le produit $A_1 A_2 A_3 \dots A_k$, où pour tout $i \in \{1, \dots, k - 1\}$, la matrice A_i est de dimension $n_i \times m_i$ et A_{i+1} est de dimension $m_i \times m_{i+1}$. Le produit de matrices est associatif, c'est-à-dire que l'on peut appliquer n'importe quel parenthésage valide et effectuer les produit dans l'ordre prescrit par ces parenthésages.

Les parenthésages ne donnent pas tous la même complexité totale. Par exemple, prenons le produit ABC , où A est de dimension 10×60 , B de dimension 60×5 et C de dimension 5×20 . Le produit $(AB)C$ nécessite

$10 \cdot 60 \cdot 5 + 10 \cdot 5 \cdot 20 = 4000$ opérations. Le produit $A(BC)$ nécessite $60 \cdot 5 \cdot 20 + 10 \cdot 60 \cdot 20 = 18,000$ opérations.

Donnez un algorithme qui retourne le nombre minimum d'opérations à effectuer pour multiplier $A_1 A_2 \dots A_k$.

Indice: considérez les $O(n^2)$ façons de séparer $A_1 A_2 \dots A_k$ en deux parties, puis combinez les paires de résultats obtenues pour garder la minimum. Mémo-risez les valeurs pour éviter de les recalculer plusieurs fois.

Soit P un parenthésage optimal. On peut supposer que si A_k n'est pas suivi d'une parenthèse fermante, alors on ajoute des parenthèses (A_k) autour de A_k . On peut donc être certain que A_k est suivie d'une parenthèse fermante. Soit A_i la matrice précédée par la parenthèse ouvrante correspondante. Il est possible que $A_i = A_k$, mais on peut supposer $A_i \neq A_1$. Par exemple, dans $A_1(A_2A_3)A_4(A_5(A_6A_7)A_8)$, on aurait $A_i = A_5$. Il s'ensuit que le parenthésage optimal est une combinaison du parenthésage optimal pour $\{A_1, A_2, \dots, A_{i-1}\}$ et du parenthésage optimal pour $\{A_i, \dots, A_k\}$. Le problème est que l'on ne connaît pas A_i . On va donc tester tous les A_i possibles.

Soit $P[i, j]$ le nombre d'opérations minimum nécessaire pour multiplier $A_i A_{i+1} \dots A_j$. On a la récurrence

$$P[1, n] = \min_{l \in \{2, \dots, k\}} (P[1, l-1] + P[l, k] + n_1 \cdot m_{l-1} \cdot m_k)$$

On peut généraliser cette récurrence à n'importe quel i, j en écrivant

$$P[i, j] = \min_{l \in \{i+1, \dots, j\}} (P[i, l-1] + P[l, j] + n_i \cdot m_{l-1} \cdot m_j)$$

Nous allons implémenter cette récurrence avec la mémorisation. Ceci donne l'algorithme suivant.

```

fonction chaineMatrices( $\{A_1, A_2, \dots, A_k\}, i, j$ )
  //calcule l'optimal pour  $A_i, \dots, A_j$ 
  //P est une variable globale, initialisée à  $\infty$ 
  si  $P[i, j] \neq \infty$  alors
    return  $P[i, j]$ 
  si  $i = j$  alors
    return 0
  mincur =  $\infty$ 
  pour  $l = i + 1..j$  faire
    val = chaineMatrices( $\{A_1, A_2, \dots, A_k\}, i, l-1$ )
    val = val + chaineMatrices( $\{A_1, A_2, \dots, A_k\}, l, j$ )
    val = val +  $n_i \cdot m_{l-1} \cdot m_j$ 
    mincur = min(mincur, val)
  fin
   $P[i, j] = \text{mincur}$ 
  return mincur

```

Exercice 7: Considérez le problème du commis voyageur. Il existe un algorithme de programmation dynamique pour résoudre le problème en temps $O(n^2 2^n)$ (ce qui est nettement mieux que l'algorithme naïf $O(nn!)$). L'idée est de choisir un sommet de départ x arbitraire et de calculer, pour chaque sous-ensemble S et pour chaque sommet $s \in S$, le coût minimum d'un chemin qui démarre à x , passe une fois par tous les sommets de S et termine à s . Donnez un algorithme en temps $O(n^2 2^n)$ pour le problème du commis voyageur.

Soit $x \in V$, qui sera notre sommet de départ (artificiel). Soit $S \subseteq V$ et $s \in S$. On dénote $D[S, s]$ la longueur minimum d'un chemin qui démarre à x , passe par chaque sommet de S exactement une fois et qui termine à s . Pour tout $v \in V$, on a $D[\{v\}, v] = f(x, v)$. Plus généralement, pour un chemin qui passe par S et termine à s , on doit démarrer de x , visiter tous les sommets de $S \setminus \{s\}$ et aller à s . On ne sait pas qui est le dernier sommet de S à être visité, alors on les essaie tous. Donc pour $|S| \geq 2$, on a la récurrence

$$D[S, s] = \min_{t \in S \setminus \{s\}} (D[S \setminus \{s\}, t] + f(t, s))$$

On n'a qu'à implémenter cette récurrence pour des ensembles de plus en plus grands.

```

fonction tsp( $G = (V, E, f)$ )
   $D[S, s] = \infty$  pour tout  $S \subseteq V$  et  $s$ 
   $x =$  sommet arbitraire de  $V$ 
  pour  $v \in V \setminus \{x\}$  faire
     $D[\{v\}, v] = f(x, v)$ 
  fin
  pour  $i = 2..n - 1$  faire
    pour chaque sous-ensemble  $S \subseteq V$  de taille  $i$  faire
      pour chaque  $s \in S$  faire
         $D[S, s] = \min_{t \in S \setminus \{s\}} (D[S \setminus \{s\}, t] + f(t, s))$ 
      fin
    fin
  fin
  return  $\min_{v \in V \setminus \{x\}} (D[V \setminus \{x\}, v] + f(v, x))$ 

```

La complexité est $O(2^n n^2)$ car il y a $O(2^n n)$ entrées dans D et chaque entrée prend un temps $O(n)$ à se calculer.